# Rapport technique développement logiciel - Shizima

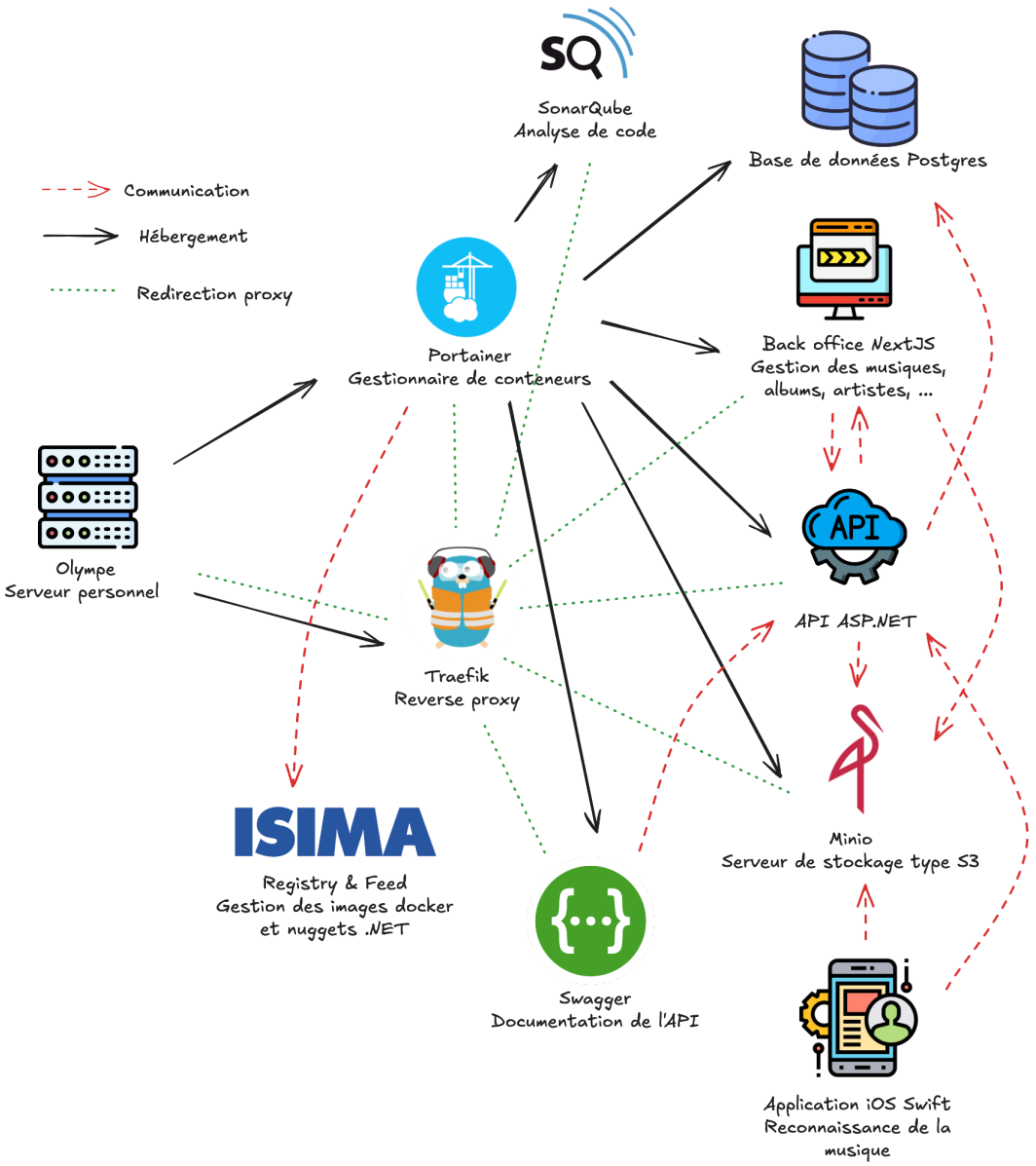
PERRET Loris
BONNEAU Baptiste
DE LA FUENTE Axel
ASTOLFI Vincent

CLERMONT AUVERGNE INP Isima

## Sommaire


| | |
|---|---|
| Infrastructure | 3 |
| Web API | 4 |
| Algorithme de reconnaissance | 6 |
| Application Mobile | 7 |
| Application Web | 8 |
| Application CLI | 9 |
| Analyse des écarts entre la conception et le développement | 10 |
| Gestion de projet et versioning | 11 |
| Annexes | 12 |

## Infrastructure



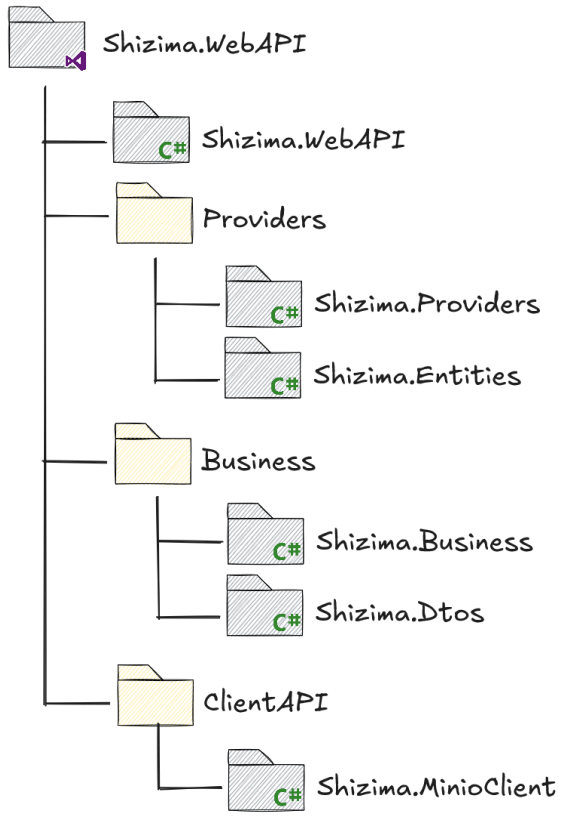

L'hébergement de nos applications suit le schéma ci-dessus.

Pour l'hébergement de nos applications, celles-ci sont d'abord dockerisées puis stockées sur la registry de l'ISIMA. Elles sont ensuite déployées via Portainer, qui permet de créer et gérer les conteneurs Docker nécessaires à leur exécution. Un reverse proxy Traefik est utilisé pour attribuer une URL publique à chaque application. L'ensemble de l'infrastructure est hébergé sur Olympe, le serveur personnel d'un membre de l'équipe Shizima. Les API sont développées avec ASP.NET et documentées grâce à Swagger. Concernant les données multimédia, les musiques et les images sont stockées sur MinIO, un service de stockage compatible S3, similaire à Amazon S3. Enfin, l'algorithme de reconnaissance musicale est fourni sous forme d'un package NuGet externe écrit en C#, et hébergé sur le feed privé de l'ISIMA. Pour finir, Sonarqube nous permet d'analyser le code et de repérer d'éventuelles erreurs.

## Web API

Shizima.WebAPI est une API RESTful développée en ASP.NET Core (C#), conçue pour fournir des services backend robustes et sécurisés. Il s'agit d'une technologie bien connue de certains membres de l'équipe ce quia permis d'avancer très rapidement et de produire un travail de qualité sur cette partie du projet. Elle s'appuie sur un ensemble de bonne pratique, permettant d'être facilement maintenue : utilisation d'Entity Framework, mapping en DTO, ainsi qu'un découpage clair en services avec injection de dépendances.

Voici comment est organisée la solution de l'API :




Shizma.WebAPI : Ce projet contient la partie Web, avec les controllers définissant les routes. C'est le cœur du projet, c'est ce dernier qu'il faut exécuter pour démarrer l'API.

Shizima.Providers : Ce projet définit le mapping des entités avec notre base de données Postgres, à l'aide d'Entity Framework.

Shizima.Entities : Ce projet définit les classes entités stockées en base de données.

Shizima.Business : Ce projet contient la logique de l'API. Ici sont stockés les services permettant de correctement gérer les appels à la base ainsi qu'à lancer les algorithmes de reconnaissance.

Shizima.Dtos : Ce projet contient les classes dtos, permettant de découpler les entités de la base de données des objets exposés via l'API. Cela évite d'exposer des champs sensibles ou inutiles.

Shizima.MinioClient : Ce projet est un client Web permettant de déposer les données type fichier (audios et images), dans sur un serveur *file-system* appelé Minio.

Le projet principal à exécuter pour lancer l'API *Shizima.WebAPI* constitue la couche exposée de l'application. Il contient les Controllers, qui définissent les différentes routes HTTP(S) (GET, POST, PUT, DELETE) accessibles par les clients, et qui orchestrent les appels aux services métiers. Chaque controller est responsable d'un domaine fonctionnel précis (ex. : UserController, AuthController, MusicController…) et utilise des annotations ([HttpGet], [Authorize], [ProduceResponse], etc.) pour définir les routes et les règles d'accès. Cette couche utilise un model différent de celle des autres couches. Ce dernier est basé sur le principe d'objets DTOs (Data Transfer Object). Ce sont des classes utilisées pour transporter des données entre le backend (API) et le client (application frontend ou mobile). Leur principal objectif est de contrôler les données exposées, en évitant de retourner directement les entités de la base de données. Afin de jongler facilement des entités de

base de données, et des objets DTOs, nous utilisons donc des mappers. Il s'agit de méthodes d'extensions de chacune des classes permettant de passer d'un objet entité à un objet DTO facilement (ToDto(), ToEntity()).

Ce projet gère également la configuration de l'authentification JWT. Lorsqu'un utilisateur s'authentifie via la route dédiée (/auth) à l'aide d'une requête POST, et qu'il fournit un identifiant et un mot de passe valide, un token JWT est généré, signé, puis retourné au client. Ce token doit être inclus dans les requêtes suivantes via l'en-tête *Authorization: Bearer*, permettant de sécuriser l'accès aux ressources protégées. Le pipeline de l'application, configuré dans *Program.cs*, intègre l'authentification et la validation du token à chaque requête, assurant ainsi une sécurité efficace et conforme aux standards du web moderne. Enfin, le controller d'authentification gère le hachage des mots de passe afin de garantir une sécurité optimale.

La partie nommée Shizima.Business contient le cœur de l'API. Il s'agit de la logique de chacune des actions que notre API permet de faire. Comme pour les controllers, cette logique est découpée en domaine fonctionnel, à travers des services (UserService, MusicService, ...). Chaque service possède des méthodes permettant d'accéder, de créer, de modifier et de supprimer des données de notre base de données (CRUD). Certains services vont aussi contenir de la logique plus poussée que de simple traitement de données de la base. Par exemple RecognitionService, ce dernier contient un certain nombre d'appels à notre librairie de traitement du signal (voir. *Algorithme de reconnaissance*) ainsi que des contrôles permettant d'assurer la cohérence de nos reconnaissances Enfin, chaque service gère les logs de ce qui est effectué avec l'API ainsi que la gestion des exceptions lors d'usages anormaux.

L'API de Shizima est développée en suivant la méthode *Code First*. Comme son nom l'indique, il s'agit de la méthode permettant aux développeurs de se plonger directement dans le code, à proprement parler, et de se concentrer uniquement sur la mise en œuvre des caractéristiques et fonctionnalités de l'application. Dans le cas de l'API, nous avons utilisé Entity Framework (EF), il s'agit d'un ORM (Object-Relational Mapper) qui permet aux développeurs .NET de gérer de manière simple, légère et extensible, des bases de données. EF permet de gérer de nombreux *providers* (SQL Server, SQLite, Postgresql, ...) de manière transparente. EF permet également de mettre à jour des bases de données (*dotnet ef migration*) et d'exécuter des requêtes sans avoir à écrire la moindre requête SQL. Vous pouvez passer par LINQ, ce qui apporte plus de lisibilité et permet au compilateur d'aider à détecter les erreurs. Dans notre cas, nous avons utilisé une base de données Postgresql.

Pour terminer, nous avons intégré Swagger grâce au package Swashbuckle.AspNetCore, qui génère automatiquement une interface utilisateur à partir des contrôleurs et annotations de l'API. Cette documentation dynamique, accessible via /swagger, facilite le développement, les tests et la collaboration entre développeurs, tout en assurant une meilleure lisibilité des routes de l'API.

*Lien vers l'API : [https://shizima-api.baptiiiste.com/swagger/](https://shizima-api.baptiiiste.com/swagger/)*

## Algorithme de reconnaissance

Shizima.CoreLib est une bibliothèque (NuGet) .NET, permettant la génération d'empreintes et leur comparaison pour de la reconnaissance audios. La solution .NET contient deux projets distincts, la bibliothèque en tant que tel (Shizima.CoreLib), ainsi qu'un CLI (client console) permettant de tester et déboguer facilement la bibliothèque. De plus, cette bibliothèque ne s'appuie sur aucune dépendance, et n'utilise que des fonctionnalités natives de .NET.

La bibliothèque contient un certain nombre de classes (dans le fichier Entities/) permettant de simplifier l'utilisation de cette bibliothèque.

Cette bibliothèque à deux points d'entrée principaux, *FingerprintGenerator.AnalyzeAudio* et *AudioRecognizer.RecognizeAudioList*.

La première permet de démarrer l'analyse d'un audio afin d'obtenir ses caractéristiques (ici appelé fingerprints ou empreintes). Cette méthode prend en paramètre un tableau de bytes, soit l'audio sous sa forme brute. Dans un premier temps l'audio va être nettoyé en le passant de stéréo à mono le cas échéant, ainsi que l'échantillonner en fonction d'une fréquence donnée. Ensuite, la classe *AudioProcessor* va être utilisé afin de générer le spectrogramme de l'audio à l'aide d'une FFT (Transformée de Fourier Rapide), qui est elle-même codé dans la classe *FFTProcessor* à l'aide de l'agorithme de Cooley-Tukey. Puis, le spectrogramme va être analysé grâce à la méthode *FingerprintGenerator.FindAnchorPoints* permettant de trouver les pics d'énergie significatifs du spectrogramme. Chaque point est associé à des valeurs temporelles et stocké dans une liste d'objets appelé *AnchorPoints*. Finalement, cette liste est hachée afin de générer notre liste de *Fingerprints*.

Le deuxième point d'entrée concerne la comparaison de ces listes de *Fingerprints.* La méthode *AudioRecognizer.RecognizeAudioList* prend en paramètre les empreintes de l'audio analysé, ainsi qu'un dictionnaire d'audio connu, avec en clé l'identifiant de notre audio, et en valeur, sa liste d'empreintes associée. Cette méthode est assez brute, elle va itérer à travers notre dictionnaire, utilisé la méthode *FingerprintMatcher.FindMatches* afin de détecter les correspondances entre deux ensembles d'empreintes, puis utilisé *FingerprintMatcher.AnalyzeMatches* pour s'assurer que les matches sont cohérents. Cette vérification se fait à travers un histogramme basé sur les décalages temporels des empreintes. En effet, une multitude de correspondances ne signifie pas que les audio sont les mêmes. Si les correspondances n'ont aucun ordre logiquetemporellement, cela n'est pas recevable. Pour finir, un objet *RecognitionResult* est renvoyé avec l'identifiant de la musique ayant le plus de correspondance, et qui n'est pas aberrant.

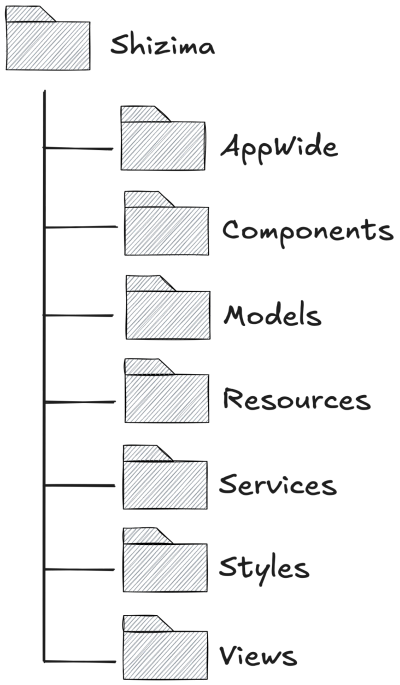
Il est aussi important de noter que toutes les méthodes "utilitaires", tel que la FFT, génération d'un spectrogramme, ect… sont toutes accessibles hors des deux points entrée présentéci-dessus Il est donc possible de facilement utiliser cette librairie que partiellement dans d'autres cadres que notre application Shizima.

Cette bibliothèque NuGet est donc publiée en version 1.0.17670, sur le registry GitLab de notre projet Shizima. Pour utiliser ce NuGet, il suffit de s'abonner au feed suivant : https://gitlab.isima.fr/api/v4/groups/11770/-/packages/nuget/index.json

## Application mobile

L'application mobile est réalisée en Swift et SwiftUI pour une application iOS. Nous avons fait le choix d'une application iOS car un membre de notre équipe maîtrisait cette plateforme. Le choix du Swift et du SwiftUI a été fait tout d'abord car c'est ces languages que le membre de notre équipe utilise. Cela permet aussi de faire une application native qui permet de meilleures performances ainsi que l'accès à des API utiles.

Le projet de l'application iOS de Shizima est découpé en plusieurs dossiers :




AppWide : Ce dossier contient les éléments globaux de l'application comme l'état et le point d'entrée de l'application.

Components : Ce dossier contient des composants réutilisables pour les vues de l'application.

Models : Ce dossier contient le model de notre application

Resources : Ce dossier contient les ressources utiles à l'application comme les couleurs, les fonts et les images.

Services : Ce dossier contient les services utiles pour l'application : le player d'audio et le scanner d'audio, le service pour les requêtes à l'API et la navigation de l'application.

Styles : Ce dossier contient des éléments de style pour les vues.

Views : Ce dossier contient les vues de l'application.

L'architecture de l'application est organisée selon le principe MVVM : Model - View - ViewModel. Grâce à cela, le plus de logique est déplacé dans les ViewModel et non dans les vues.

Lors du lancement de l'application, on récupère de l'API l'historique de recherche de l'utilisateur pour le stocker dans la base de données locales dans le téléphone. On travaille ensuite avec les éléments en base. Cela permet de continuer de pouvoir lire des musiques malgré le manque de connexion internet.

Lors des appels API, on travaille avec des objets DTO que l'on transforme ensuite en objet de notre model.

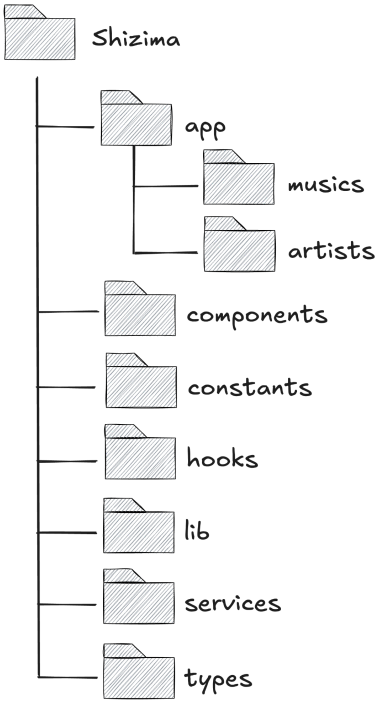
Une gestion des erreurs est aussi mise en place lors des différents appels à l'API ou à la base de données afin de ne pas bloquer l'utilisation de l'application.

Il y a deux vues principales à l'application. L'écran de scan et l'écran d'historique. autres vues comme le player d'une musique et la page d'un artiste sont utilisées pour enrichir l'expérience utilisateur.

## Application web

Le back office permettant la gestion des artistes, musiques, albums, … a été développé en typescript avec le framework NextJS qui permet un développement rapide, robuste et facilement maintenable.

Le projet suit l'architecture suivante :




App: Un dossier pour la gestion des pages qui permet, grâce à NextJS, de faire un routage automatique. Par exemple, créer un dossier "artists" avec un fichier "page.tsx" à l'intérieur créera la route "/artists". NextJS permet aussi de faire du routage avec des paramètres en créant un dossier entre crochet :
pour faire une route "/artists/{id}", il faudra créer un dossier [id] dans le dossier "artists".

Components: Un dossier divisé en 3 parties regroupant les composants utilisés dans les pages :
"shared" pour les composantscommuns à plusieurs pages
"ui" pour les composants importés de librairies externes
"pages" pour les composants spécifiques à une page

Lib: Un dossier qui regroupe toutes les fonctions liées aux librairies externes

Services : Un dossier pour gérer les appels à l'API

Le fonctionnement est plutôt simple, lors de l'arrivée sur une page, les données que nous souhaitons afficher seront récupérées de l'API et seront passées aux composants enfants selon les besoins.

Nous avons utilisé la librairie de composant "Shadcn" que nous avons modifié et ajusté pour la partie UI et nous gérons les formulaires avec Zod. L'authentification est gérée avec NextAuth, qui utilise des cookies sécurisés pour stocker les sessions côté client. À chaque requête, un middleware vérifie la session active via les cookies, et protège les routes en redirigeant les utilisateurs non authentifiés. Les appels à l'API incluent automatiquement les informations de session pour s'assurer que seules les actions autorisées sont possibles.

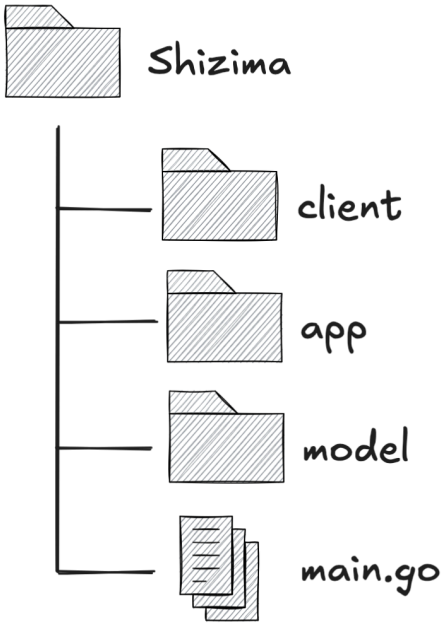
L'architecture des composants a été faite et pensée pour simplifier au maximum l'ajout de nouvelles pages en créant des composants réutilisables, modulables et génériques.

L'application se concentre principalement autour de la page "Artiste" qui récupère les informations de l'artiste, ses albums et ses musiques. Il y a la possibilité pour chacun de les modifier, supprimer ou d'en créer de nouveaux.

Nous sommes partis sur une pagination gérée côté backend afin d'avoir de meilleures performances car cela nous évite de charger toutes les données d'un coup, nous nous contentons juste de récupérer celles à afficher

## Application CLI

Dans le cadre de ce projet, une interface en ligne de commande (CLI) a été développée en langage Go. Son objectif principal est de permettre de tester les fonctionnalités de reconnaissance musicale de l'application sans nécessiter un appareil iOS. Cela offre une solution pratique pour les phases de développement, de débogage, ou pour les utilisateurs ne disposant pas d'environnement mobile Apple.




Le choix du langage Go s'est imposé naturellement en raison de sa simplicité, de sa rapidité d'exécution et de son écosystème riche. En particulier, la bibliothèque Bubbletea a été utilisée pour créer une interface terminale (package app). Cette bibliothèque permet de structurer une application CLI selon un modèle déclaratif inspiré des architectures Elm, favorisant une organisation claire du code et une grande flexibilité dans l'affichage.`

L'application CLI interagit directement avec l'API (package app) via la route /recognition/wavFile, en permettant à l'utilisateur de sélectionner un fichier audio local, de lancer une analyse, et de consulter les résultats. L'architecture du projet est modulaire : la gestion des menus, de l'affichage des résultats et des appels HTTP est séparée dans des fichiers distincts (menu.go, resultViewer.go, etc.). Les données récupérées via l'API sont transformées à l'aide de classe DTO présente dans le package model afin de pouvoir être affiché dans l'interface du terminal.

Au-delà de son utilité directe, cette CLI illustre pleinement la flexibilité de notre architecture. Le fait de pouvoir créer facilement un nouveau client autonome, tout en réutilisant exactement la même API backend, démontre que notre système est conçu pour être agnostique au type de plateforme. Qu'il s'agisse d'un terminal, d'un mobile iOS, d'un site web ou potentiellement d'une application desktop, notre infrastructure permet le développement de clients multiples, avec un minimum d'effort côté serveur. Cela ouvre la voie à une extensibilité importante et confirme la solidité de l'architecture globale du projet Shizima.

## Analyse des écarts entre la conception et le développement

*Les diagrammes non mentionnés sont considérés comme respectés.*

Diagramme de classe :

- Suppression Mp3ToWavConverte et IAudioConverter: nous avons fait le choix de ne travailler qu'avec le format WAV, en interdisant l'utilisation du format MP3. Cette décision vise à garantir l'homogénéité de notre traitement audio et à nous assurer que tous les fichiers sont encodés dans un format.

MLD:

- Ajout d'une entité Administrator, afin de sécuriser certaines routes de l'API. À présent, seules les administrateurs peuvent modifier des données. Les utilisateurs simples n'ont accès qu'aux routes de lecture des données.
- Ajout d'une relation entre les musiques et les artistes, ce qui nous semblait avoir plus de sens.

En résumé, nous avons réussi à suivre la conception initialement proposée avec une grande fidélité. Les principaux changements apportés ne découlent ni de contraintes techniques, ni de limitations en termes de connaissances. Dans la plupart des cas, les ajustements effectués relèvent de choix stratégiques, par exemple, l'utilisation d'un seul format de compression audio pour maximiser les performances de l'algorithme de reconnaissance. D'autres modifications ont été motivées par des considérations logiques, comme le fait qu'un morceau de musique ne partage pas nécessairement le même artiste que l'album auquel il appartient.

## Gestion de projet et versioning

Dans le cadre de notre projet, nous avons adopté une gestion structurée en nous appuyant sur GitLab, qui nous a permis d'assurer le versioning du code ainsi qu'une organisation claire des repos. Chaque fonctionnalité ou composant majeur était associé à un dépôt dédié lui-même dans un sous-groupe en fonction du rôle (applicatif, serveur ou bibliothèque), ce qui a facilité la maintenance et le travail en parallèle.

Nous avons mis en place des pipelines CI/CD afin d'automatiser les tests et le déploiement, garantissant ainsi une intégration continue et un code de qualité. De plus, la mise en place de SonarQube au sein de nos pipelines nous a permis d'avoir un suivi sur la qualité du code produit sur chacun des projets (sauf l'application iOS, car l'analyse du langage Swift nécessite la version payante de SonarQube).

Pour favoriser la collaboration, nous avons suivi des conventions de nommage claires et utilisé activement le système d'issues et de labels pour organiser nos tâches, suivre les évolutions, et prioriser les corrections.

Le projet a été conduit selon une méthode agile, avec des réunions hebdomadaires (weekly) permettant de faire le point sur l'avancement, ajuster les objectifs et assurer une répartition équilibrée des tâches. Cette approche nous a permis de rester flexibles et efficaces tout au long du développement.

## Annexes

Gitlab: https://gitlab.isima.fr/shizima
Swagger: https://shizima-api.baptiiiste.com/swagger/
API: https://shizima-api.baptiiiste.com
Back-office: https://shizima.baptiiiste.com (shizima@test.com, shizima123)
Minio: https://minio.baptiiiste.com (minio, password)
Sonarqube: https://sonarqube.baptiiiste.com
Adminer: https://shizima-db.baptiiiste.com (System: PostgreSQL, Server: shizima_postgres, Username: shizima, Password: 175za*ppehf87+, Database: shizima_db)